

✓ Gradient Decent Theory

$$h(x) = w * x + b$$

here, b is the bias or intercept x represents the feature vector w represents the weight vector or slope

$$\frac{1}{m} \sum_{i=1}^m (y^{(i)} - h(x^{(i)}))^2$$

Loss Function = 1/n summation (actual y - predicted y)

we have to minimize this loss function by choosing the best values of slope(w) and intercept(b)

✓ Pseudo Code

Gradient descent: Pseudo Code:

Start with some w Keep changing w to reduce J(w) [Loss Function] until we hopefully end up at a minimum. Algorithm:

repeat until convergence { tmpi = wi - alpha * dwi wi = tmpi } OR

Updating Parameter: { m(new) = m (old) - learning rate (derivative function with respect to m or with respect to b

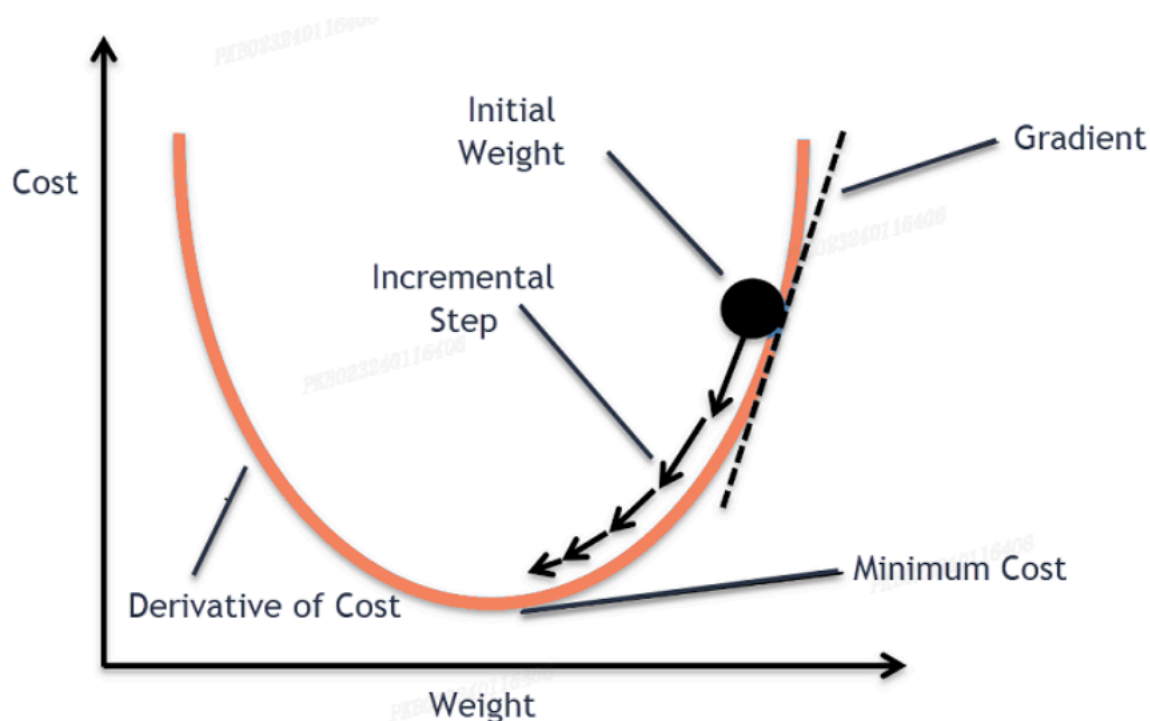
1. For the slope *m* (the coefficient of *x*):

$$m_{\text{new}} = m_{\text{old}} - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

2. For the intercept *b*:

$$b_{\text{new}} = b_{\text{old}} - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})$$

where alpha is the learning rate.



✓ Let's Begin with Implementation

```
# Importing Required Libraries
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
pd.set_option('display.max_columns',100)    # to display max 100 columns not as per the default set in the pandas
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
plt.rcParams['font.size'] = 12               # to set the font size for text elements
plt.rcParams['figure.figsize'] = (22, 5)    # to set the figure size for each plot or figure that we will use
plt.rcParams['figure.dpi'] = 100

import sqlite3
from sklearn.metrics import r2_score, roc_curve, auc           #Evaluation metrics
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer

from urllib.request import urlretrieve                        #to load the dataset from the provided url efficiently

medical_dataset_url = 'https://raw.githubusercontent.com/JovianML/opendatasets/master/data/medical-charges.csv'

# storing the data into dataframe
medical_df = pd.read_csv(medical_dataset_url)
medical_df.head(10)
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
5	31	female	25.740	0	no	southeast	3756.62160
6	46	female	33.440	1	no	southeast	8240.58960
7	37	female	27.740	3	no	northwest	7281.50560
8	37	male	29.830	2	no	northeast	6406.41070
9	60	female	25.840	0	no	northwest	28923.13692

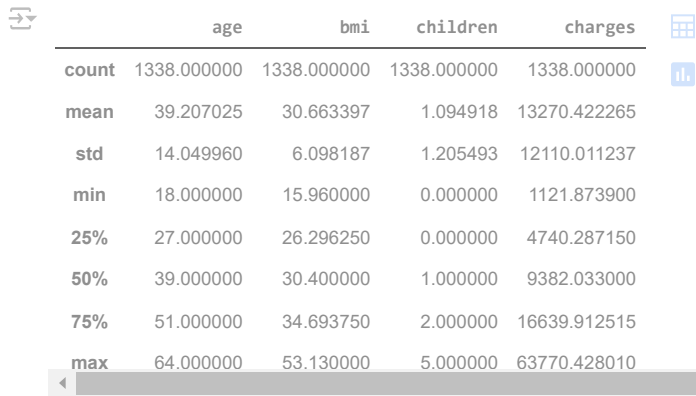
Next steps:

[Generate code with medical_df](#)[View recommended plots](#)[New interactive sheet](#)

```
# checking the shape of the dataset
medical_df.shape
```

```
(1338, 7)
```

```
medical_df.describe()
```



	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

```
medical_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    age         1338 non-null   int64
1    sex         1338 non-null   object
2    bmi         1338 non-null   float64
3    children    1338 non-null   int64
4    smoker      1338 non-null   object
5    region      1338 non-null   object
6    charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

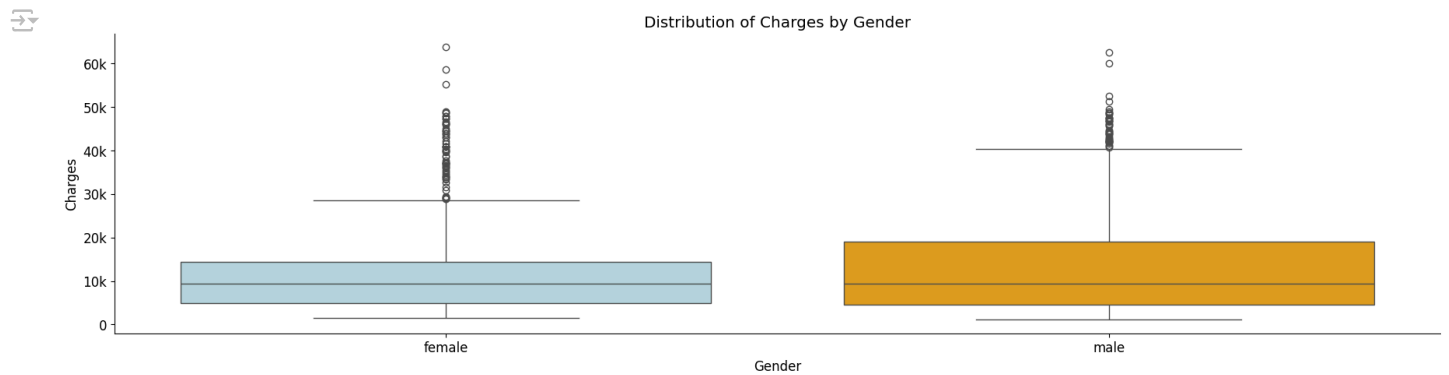
✓ Exploratory Data Analysis (EDA)

```
# Creating sql database
conn = sqlite3.connect('medical_database.db')

# Creating Dataframe into table
medical_df.to_sql('medical_data', conn, if_exists='replace', index=False)
```

```
1338
```

```
custom_palette = {'female':'lightblue', 'male':'Orange'}
sns.boxplot(medical_df, x='sex', y='charges', palette=custom_palette) # After running this snippet we would know that it shows simple box
yticks = plt.gca().get_yticks() #approaching or getting y ticks or y labels
y_labels = [f"{round(i/1000)}k" if i != 0 else '0' for i in yticks] #formating y labels like 1000 to 1k
plt.gca().set_yticklabels(y_labels)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.title('Distribution of Charges by Gender')
plt.xlabel('Gender')
plt.ylabel('Charges')
plt.figure(figsize=(8,5))
plt.show()
```



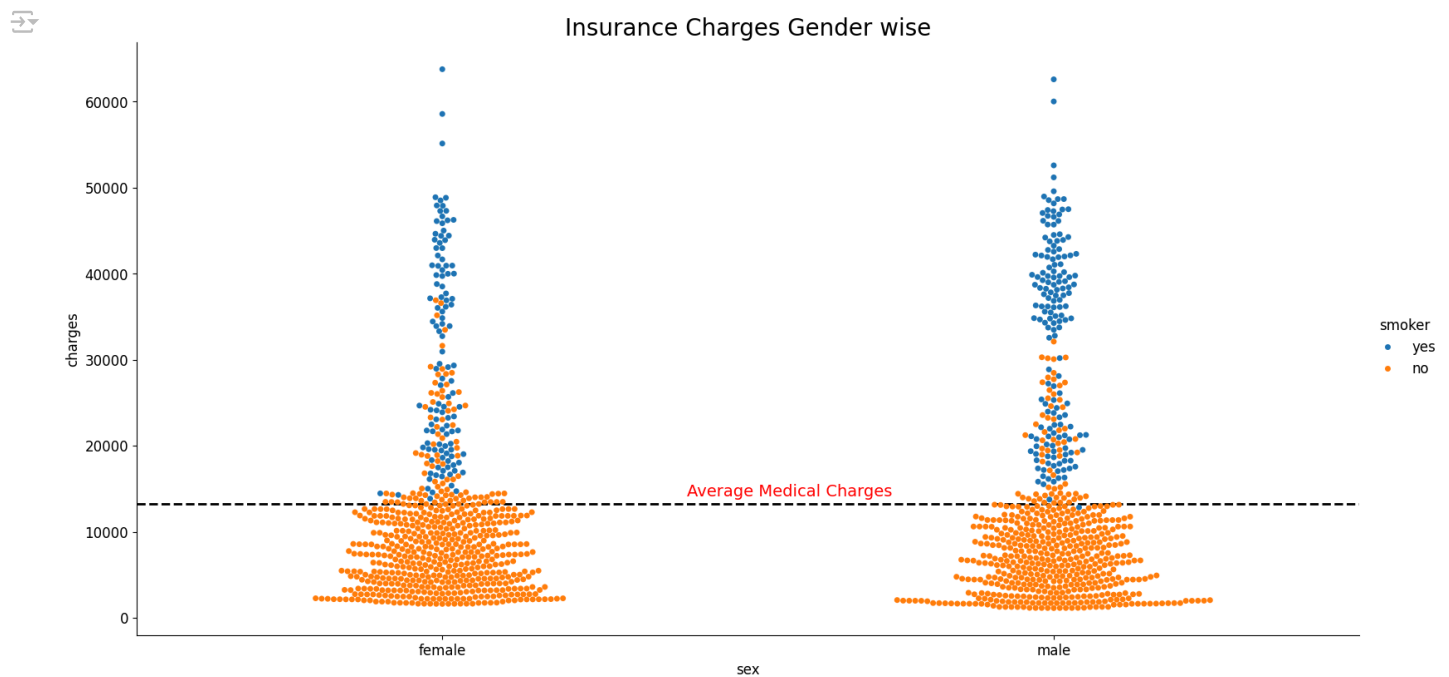
<Figure size 800x500 with 0 Axes>

From the above plot we can see that males are about to spend more insurance charges then female

```
sns.catplot(data=medical_df, x = 'sex', y = 'charges', kind = 'swarm', hue = 'smoker', aspect = 2, height = 8)
```

```
# plotting mean line of charges in graph
plt.axhline(medical_df['charges'].mean(), linestyle = '--', lw=2,color='black')
plt.annotate(f'Average Medical Charges', (.4, medical_df['charges'].mean()+900), fontsize=14, color='red')
plt.title('Insurance Charges Gender wise', fontsize=20)
plt.show()

# plt.axhline(medical_df['charges'].mean(), linestyle='--', lw=2, zorder=1, color='black')
```

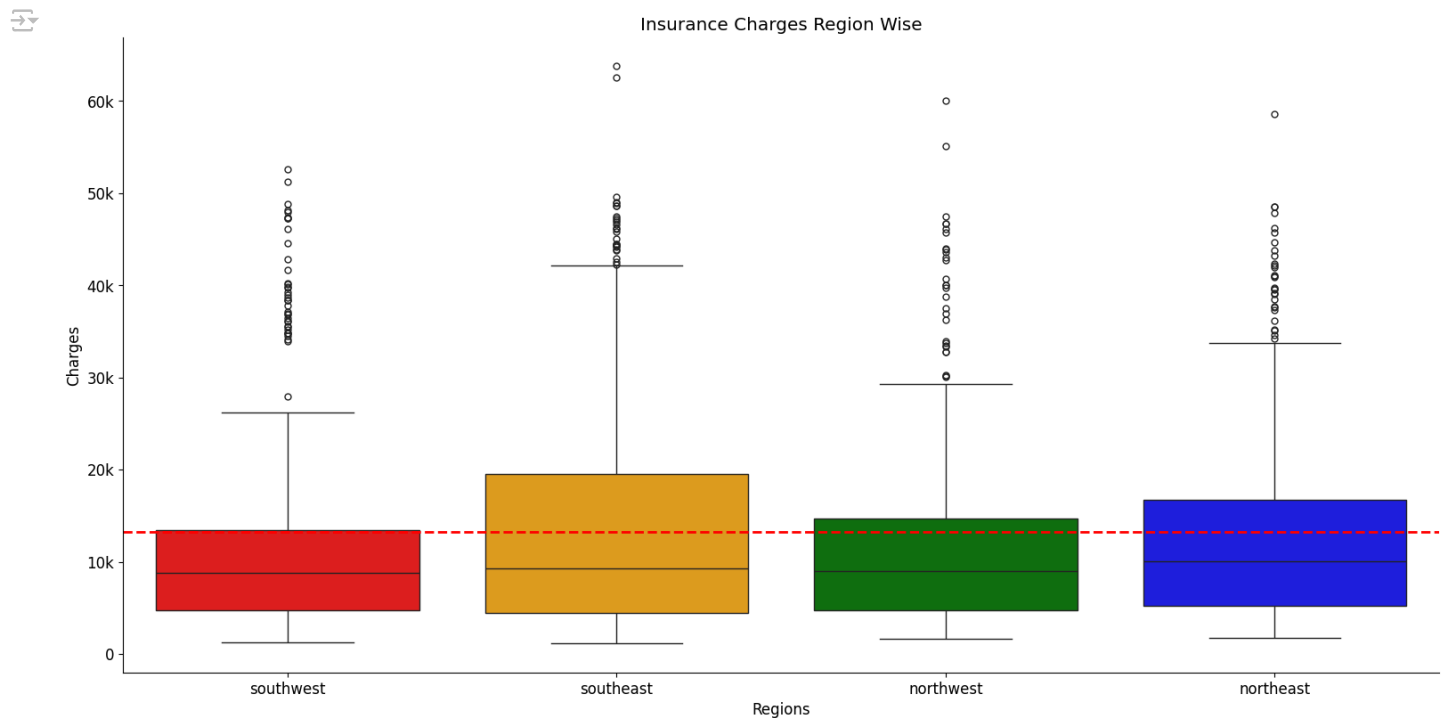


From the plot above we can see that male smokers are likely to spend more on insurance

```

custom_palette = ['red', 'orange', 'green', 'blue']
sns.catplot(data=medical_df, x='region', y='charges', kind='box', aspect=2, height=8, palette=custom_palette)
yticks=plt.gca().get_yticks()
y_labels=[f'{round(i/1000)}k' if i!=0 else '0' for i in yticks]
plt.gca().set_yticklabels(y_labels)
plt.axhline(medical_df['charges'].mean(), linestyle='--', lw=2, color='red')
plt.title('Insurance Charges Region Wise')
plt.xlabel('Regions')
plt.ylabel('Charges')
plt.show()

```

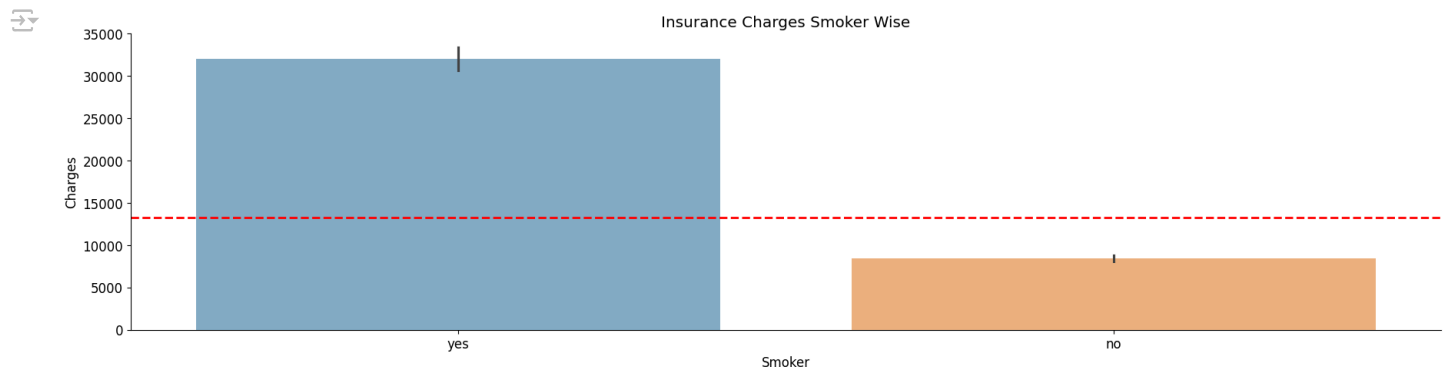


Insurance charges spend by more in southeast region as compare to northwest or southwest

```

sns.barplot(data=medical_df, x='smoker', y='charges', hue='smoker', alpha=0.6, dodge=False)
plt.axhline(medical_df['charges'].mean(), linestyle='--', lw=2, color='red')
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.title('Insurance Charges Smoker Wise')
plt.xlabel('Smoker')
plt.ylabel('Charges')
plt.show()

```



Smokers paying more insurance charges than non smokers

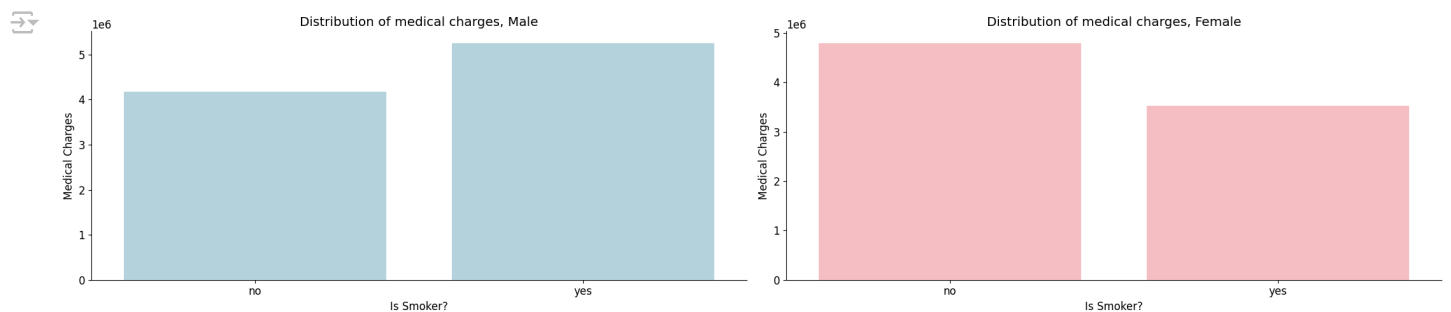
```
# Query to get data from our database that we created above as medical_data
query = """
select sex, smoker, round(sum(charges), 2) as medical_charges
from medical_data
group by sex, smoker
"""

temp = pd.read_sql(query, conn)          #loading data from sql to pandas dataframe
male_df = temp[temp['sex']=='male']      #filtering of sex column to male only
female_df = temp[temp['sex']=='female']  #filtering of sex column to female only

# Creating subplots of male and female with respect to the smokers or not
plt.subplot(1,2,1)                       #first subplot
sns.barplot(x=male_df['smoker'], y=male_df['medical_charges'], color='lightblue')
plt.xlabel('Is Smoker?')
plt.ylabel('Medical Charges')
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.title('Distribution of medical charges, Male')

plt.subplot(1,2,2)                       #second subplot
sns.barplot(x=female_df['smoker'], y=female_df['medical_charges'], color='lightpink')
plt.xlabel('Is Smoker?')
plt.ylabel('Medical Charges')
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.title('Distribution of medical charges, Female')

plt.tight_layout()                       #adjusting layout
plt.show()
```



From the above chart we can see that Male who smoke pay high charges while female who don't smoke pay high charges

```

query = """
select
  case
    when age<30 then 'Young'
    when age<60 then 'Adult'
    else 'Old'
  end as age_group,
  round(avg(charges),2) as avg_charges

```

```

From
  medical_data
where smoker == 'yes'
group by
  case
    when age<30 then 'Young'
    when age<60 then 'Adult'
    else 'Old'
  end;
"""

```

```

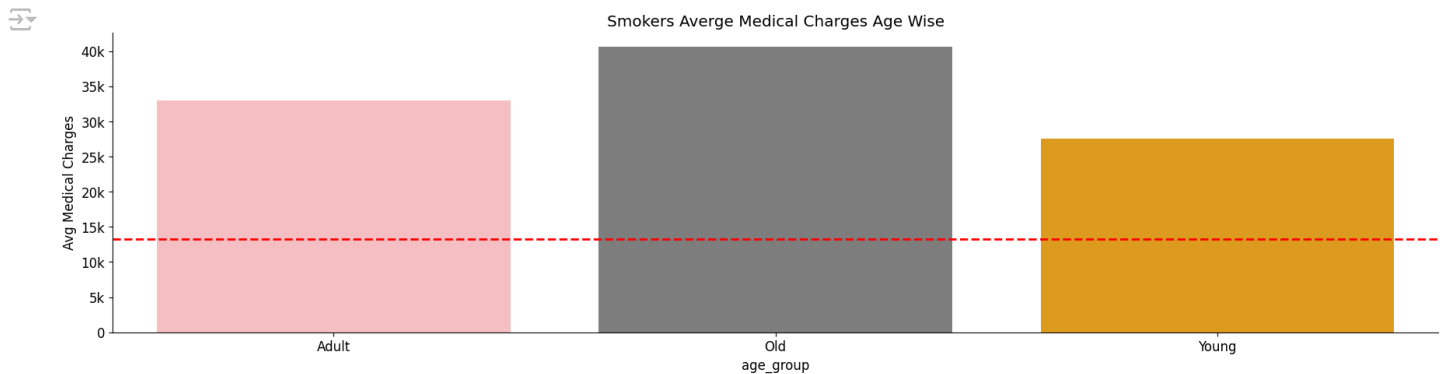
temp = pd.read_sql(query, conn)
custom_palette = ['lightpink', 'grey', 'orange']

```

```

sns.barplot(data=temp, x='age_group', y='avg_charges', hue='age_group', dodge=False, palette=custom_palette)
yticks=plt.gca().get_yticks()
y_labels = [f"{round(i/1000)}k" if i != 0 else "0" for i in yticks]
plt.gca().set_yticklabels(y_labels)
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)
plt.axhline(medical_df['charges'].mean(),linestyle='--', lw=2, zorder=1, color='red')
# plt.annotate(f'Avg Medical Charges', (.5, medical_df['charges'].mean()+1000), fontsize=12, color='darkred')
plt.title('Smokers Average Medical Charges Age Wise')
plt.ylabel('Avg Medical Charges')
plt.show()

```



```

query = """
select
  case
    when age<30 then 'Young'
    when age<60 then 'Adult'
    else 'Old'
  end as age_group,
  round(avg(charges),2) as avg_charges

```

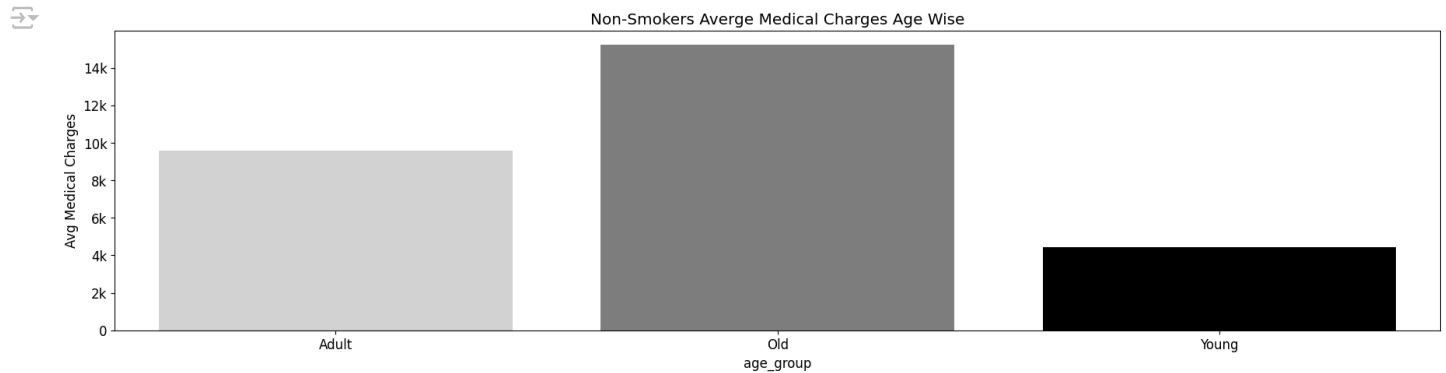
```

From
  medical_data
where smoker == 'no'
group by
  case
    when age<30 then 'Young'
    when age<60 then 'Adult'
    else 'Old'
  end;
"""

```

```
temp = pd.read_sql(query,conn)
custom_palette = ['lightgrey','grey','black']

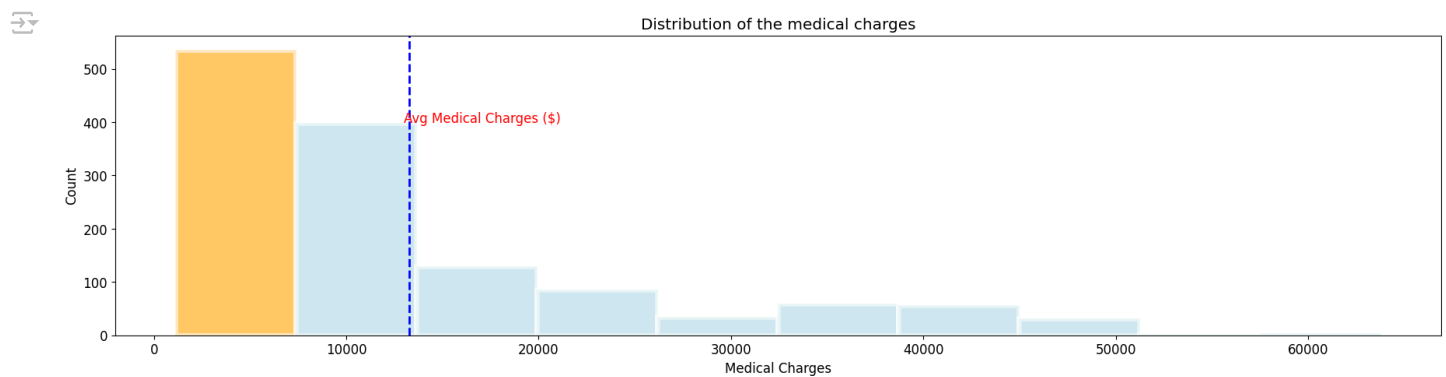
sns.barplot(data=temp, x='age_group', y = 'avg_charges', hue='age_group', dodge = False, palette=custom_palette)
yticks = plt.gca().get_yticks()
y_labels = [f"{round(i/1000)}k" if i != 0 else "0" for i in yticks]
plt.gca().set_yticklabels(y_labels)
plt.title('Non-Smokers Average Medical Charges Age Wise')
plt.ylabel('Avg Medical Charges')
plt.show()
```



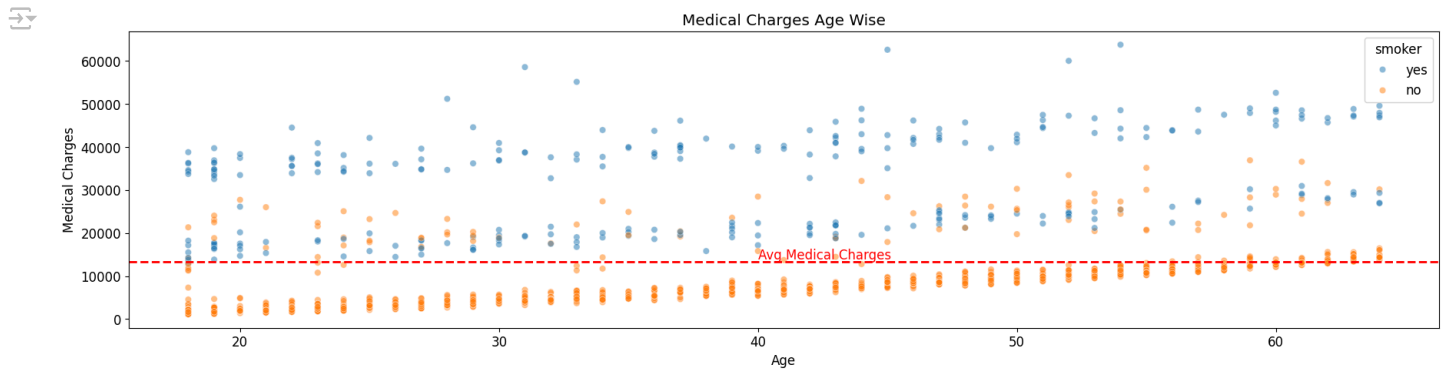
Adults and young who smoke spend more on insurance charges as compare to the non- smoker Adults and young

```
fig , ax = plt.subplots()
N, bins, patches = ax.hist(np.array(medical_df.charges), edgecolor = 'white', color = 'lightblue', linewidth = 5, alpha=0.6)

for i in range(0,1):
    patches[i].set_facecolor('orange')
plt.title('Distribution of the medical charges')
plt.axvline(medical_df['charges'].mean(), linestyle='--', lw=2, color = 'blue', zorder = 1)
plt.annotate(f"Avg Medical Charges ($)", (13000,400),color = 'red', fontsize = 12)
plt.xlabel('Medical Charges')
plt.ylabel('Count')
plt.show()
```

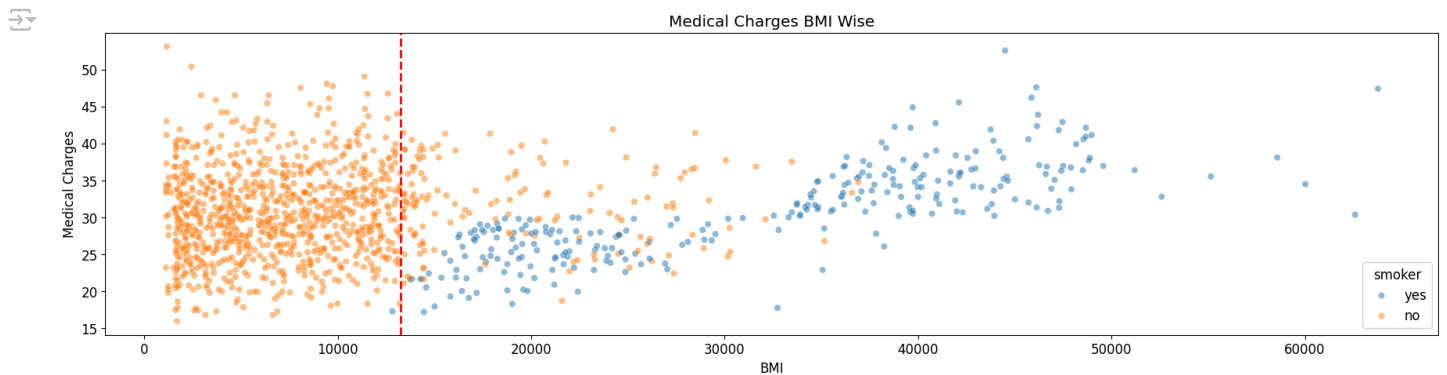



```
sns.scatterplot(y=medical_df['charges'], x= medical_df['age'], hue = medical_df['smoker'], alpha = 0.5)
plt.axhline(medical_df['charges'].mean(),linestyle='--', lw = 2, zorder = 1, color = 'red')
plt.annotate(f"Avg Medical Charges", (40,13900), color='red', fontsize = 12)
plt.title('Medical Charges Age Wise')
plt.xlabel('Age')
plt.ylabel('Medical Charges')
plt.show()
```

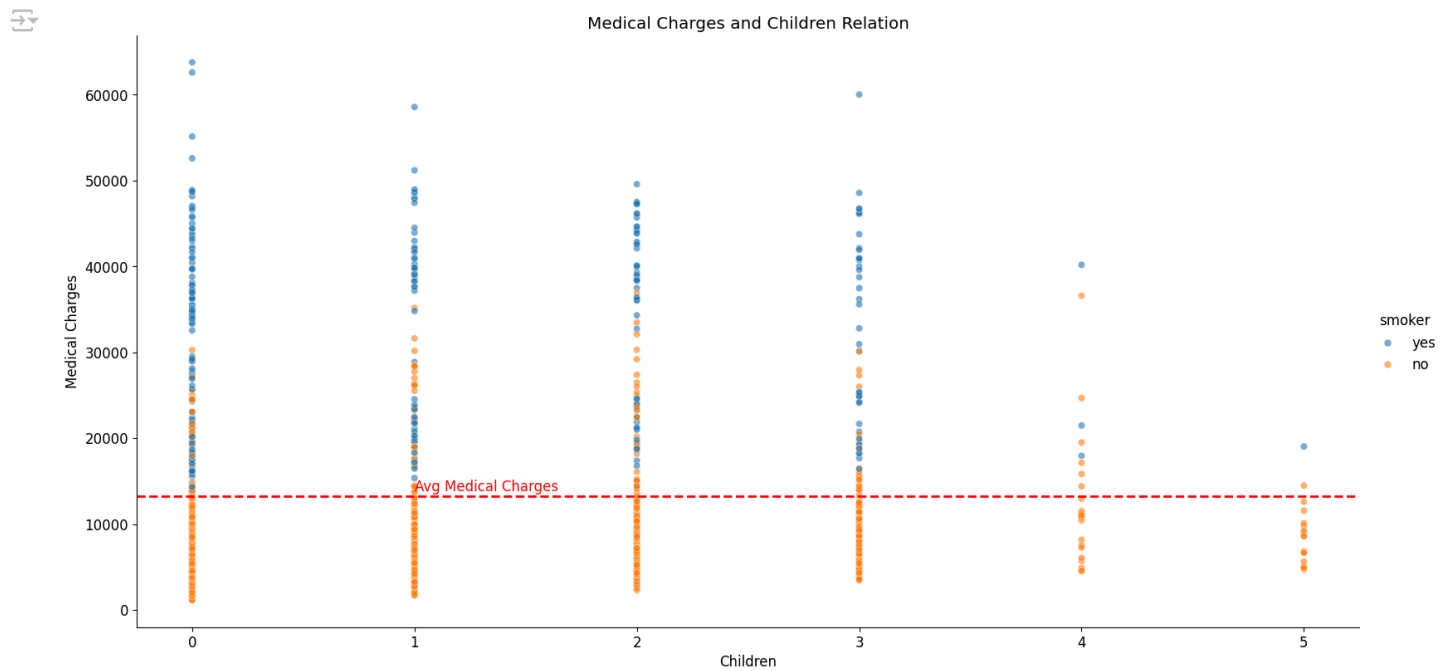


Smoker are likely to spend more on insurance charges as compare to non-smoker

```
sns.scatterplot(y= medical_df['bmi'], x= medical_df['charges'], hue = medical_df['smoker'], alpha=0.5)
plt.axvline(medical_df['charges'].mean(),linestyle='--', lw=2, zorder=1, color='red')
plt.annotate(f"Avg Medical Charges", (13900,400), color='red', fontsize=12)
plt.title('Medical Charges BMI Wise')
plt.xlabel('BMI')
plt.ylabel('Medical Charges')
plt.show()
```

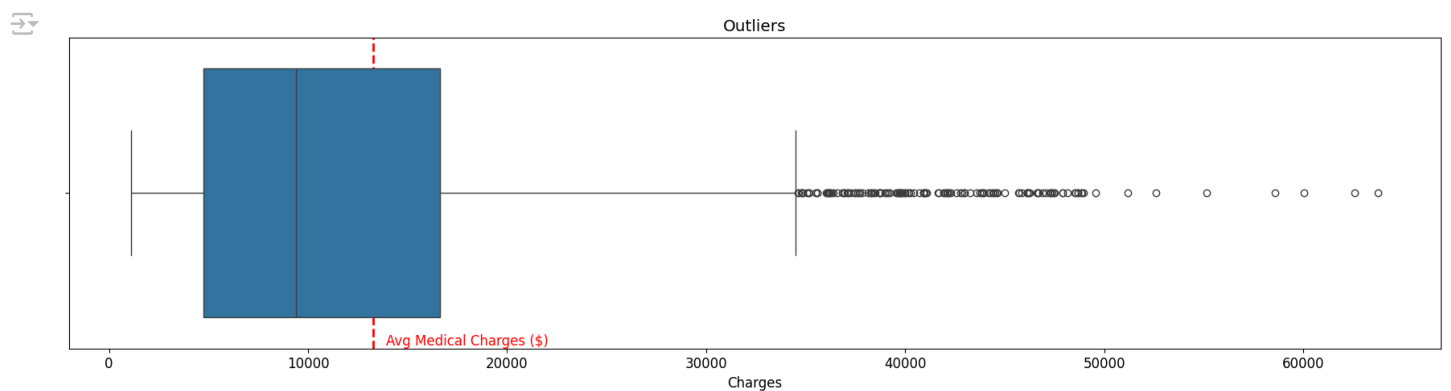


```
sns.relplot(data=medical_df, x= medical_df['children'], y=medical_df['charges'], hue=medical_df['smoker'],alpha=0.6, aspect=2, height=8)
plt.axhline(medical_df['charges'].mean(),linestyle='--',lw=2, color='red')
plt.annotate(f"Avg Medical Charges", (1,13900),color='red', fontsize=12)
plt.title('Medical Charges and Children Relation')
plt.xlabel('Children')
plt.ylabel('Medical Charges')
plt.show()
```



people with 0, 1 and 3 childrens are more likely to spend more insurance charges

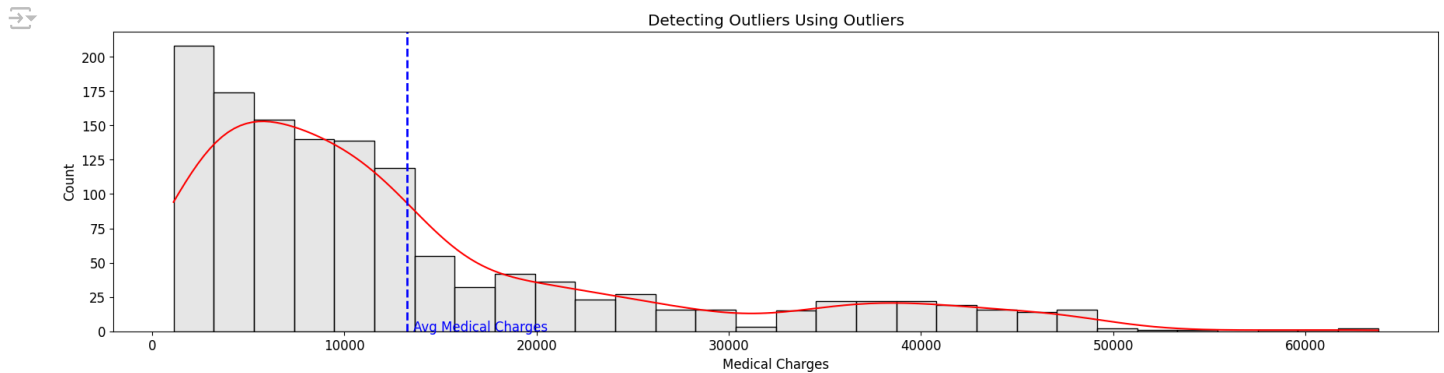
```
sns.boxplot(x=medical_df['charges'])
plt.axvline(medical_df['charges'].mean(), color='red', linestyle='--', zorder=1, lw=2)
plt.annotate(f'Avg Medical Charges ($)', (13900, 0.49), color='red', fontsize=12)
plt.title('Outliers')
plt.xlabel('Charges')
plt.show()
```



✓ Data Pre-Processing

```
ax=sns.histplot(medical_df['charges'], kde=True, color='lightgrey')
ax.lines[0].set_color('red')
```

```
plt.axvline(medical_df['charges'].mean(), linestyle='--', lw=2, color='blue')
plt.annotate(f'Avg Medical Charges', (13600, 0.40), color='blue', fontsize=12)
plt.title('Detecting Outliers Using Outliers')
plt.xlabel('Medical Charges')
plt.ylabel('Count')
plt.show()
```



```
data = sorted(medical_df['charges'].values)          # extracting values from the medical_df dataframe and sorted them

data_mean = np.mean(data)                          #calculating mean of the data
data_std = np.std(data)                            #calculating standard deviation of the data

cut_off = data_std*3                                #calculating cut_off value which is 3 times of the standard deviation

lower = data_mean-cut_off                            #calculating lower bound and upper bound to identify potential outliers
upper = data_mean+cut_off

print('Cut_off = ', round(cut_off, 3))
print('Lower Bound = ', round(lower,3))
print('Upper Bound = ', round(upper,3))

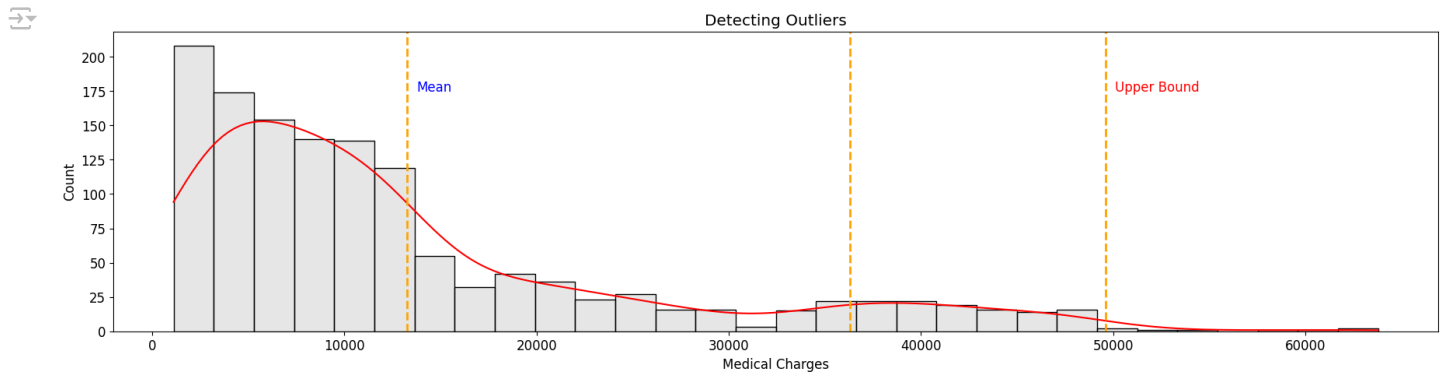
Cut_off = 36316.455
Lower Bound = -23046.033
Upper Bound = 49586.877
```

```
ax=sns.histplot(medical_df['charges'],kde=True, color='lightgrey')
plt.axvline(data_mean, linestyle='--', lw=2, color='orange')
plt.annotate(f'Mean', (data_mean+500, 175), fontsize=12, color='blue')
ax.lines[0].set_color('red')

plt.axvline(cut_off, linestyle='--', lw=2, color='orange')
plt.annotate(f'Lower Bound', (lower+500, 175), fontsize=12, color='red')

plt.axvline(upper, linestyle='--', lw=2, color='orange')
plt.annotate(f'Upper Bound', (upper+500, 175), fontsize=12, color='red')

plt.title('Detecting Outliers')
plt.xlabel('Medical Charges')
plt.ylabel('Count')
plt.show()
```



```
medical_df = medical_df[medical_df['charges'] < upper]
medical_df = medical_df[medical_df['charges'] > lower]

print('Shape of data frame after removing Outliers', medical_df.shape)
```

Shape of data frame after removing Outliers (1331, 7)

```
# Making copy of the data frame
df = medical_df.copy()
```

One Hot Encoding

```
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
categorical_cols = df.select_dtypes(include='object').columns
encoder.fit(df[categorical_cols])
```

OneHotEncoder

```
OneHotEncoder(handle_unknown='ignore', sparse_output=False)
```

```
onehot = encoder.transform(df[categorical_cols])
encoder.categories_
```

```
[array(['female', 'male'], dtype=object),
 array(['no', 'yes'], dtype=object),
 array(['northeast', 'northwest', 'southeast', 'southwest'], dtype=object)]
```

```
encoded_cols = [['female', 'male', 'smokerno', 'smokeryes', 'northeast', 'northwest', 'southeast', 'southwest']]
df[['female', 'male', 'smokerno', 'smokeryes', 'northeast', 'northwest', 'southeast', 'southwest']] = onehot
df.drop(categorical_cols, axis=1, inplace=True)
```

```
df.head()
```

	age	bmi	children	charges	female	male	smokerno	smokeryes	northeast	northwest	southeast	southwest
0	19	27.900	0	16884.92400	1.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0
1	18	33.770	1	1725.55230	0.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0
2	28	33.000	3	4449.46200	0.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0
3	33	22.705	0	21984.47061	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0
4	32	28.880	0	3866.85520	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0

Next steps:

[Generate code with df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
# Scaling
scaler = MinMaxScaler()
```

```

y = df['charges'].values

x = df.drop('charges', axis=1).values          #Dropping charges column from the dataframe to create feature matrix

print('Shape of the independent variables is: ', x.shape)
print('shape of targert variable is: ', y.shape)

↗ Shape of the independent variables is: (1331, 11)
  shape of targert variable is: (1331,)

```

✓ Derivation of the Equation

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

We will do the partial differentiation of this matrix with respect to m and b as well after putting the values $y = mx+b$ in the above equation in the place of $\hat{y}$. Our final equations after differentiation will look like this

Final derivative

$$\frac{\partial y}{\partial x_b} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\frac{\partial y}{\partial x_w} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x$$

Updating parameters Rule that we defined before

1. For the slope m (the coefficient of x):

$$m_{\text{new}} = m_{\text{old}} - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

2. For the intercept b :

$$b_{\text{new}} = b_{\text{old}} - \alpha \cdot \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})$$

✓ Building Model and Training

```

# Function for early stopping critereaa by comparing consecutive losses
def early_stopping(loss):
    for i in range(1, len(loss)):
        yield (loss[i-1], loss[i])

# Function for determining number of batches based on the size of the dataset
def batch_size(batchsize, X):
    batches = round(X.shape[0] // batchSize)
    return batches

# Gradient decent function for Linear Regression
def gradient_decent(X_train, y_train, m, b):
    yhat = np.dot(X_train,m)+b          # y = mx + b

# Mean Square Error
MSE = np.sum((y_train - yhat)**2)/N

```

```

# R square Score
r_squared = r2_score(y_train, yhat)

# Final derived equations calculations

partial_derivative_b = (-2/N) * np.sum(y_train - yhat)          #First of equation of final derivative that is given above
partial_derivative_m = -(2/N) * np.dot((y_train - yhat), X_train) # second equation of final derivative that is given above

# Updating parameters m and b using the above two equations

m = m - (learning_rate * partial_derivative_m)
b = b - (learning_rate * partial_derivative_b)

return m, b, MSE, r_squared

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.20, random_state = 50)

# Scaling the values
scaled_X_train = scaler.fit_transform(X_train)
scaled_X_test = scaler.transform(X_test)

```

$$\text{new_learning_rate} = \frac{1}{1+\text{decay_rate}} \times \text{current_learning_rate}.$$

```

# setting and fixing seed to generate same random numbers each time we run the code
np.random.seed(42)

# Total number of data points
N = x.shape[0]

# Setting initial learning rate, decay rate and other variables
learning_rate = 0.02
decay_rate = 0.01
LR = []          #list to store learning rate
Validationloss = [] #list to store validation loss
Trainingloss = [] #list to store training loss
batchsize = 30
R_squared_train = []
R_squared_test = []

# Initializing intercept and slope
Intercept = [] #list to store intercept b
Slope = [] #list to store slope m
m = np.ones(x.shape[1]) #initializing value for m and b
b = 1
print('Initial value of m and b are ', m, ', ', b)

# calculating number of batches
batches = batch_size(batchsize, x)

num_iterations = 2000

for i in range(num_iterations): #Loop for multiple iterations
    iteration = i

    for j in range(batches): #Loop for batches
        if i == 0: #First Iteration, initializing values
            if j % batchsize == 0: #This will be implemented for each batchsize
                learning_rate = learning_rate #Initializing Learning rate
                np.random.seed(42)
                np.random.shuffle([scaled_X_train, y_train])
                m, b, MSE, r_squared = gradient_descent(scaled_X_train, y_train, m, b)
                m_test, b_test, MSE_test, r_squared_test = gradient_descent(scaled_X_test, y_test, m, b)
            else:
                m = m
                b = b

```

```

Initial value of m and b are [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. ], 1
Iteration: 0/2000 [-----] Loss: 2.21e+08 Validation_Loss: 5.201340e+07 r_squared: -1.0383 r_squared_test: -0.8267
Iteration: 100/2000 [-----] Loss: 4.85e+07 Validation_Loss: 1.243413e+07 r_squared: 0.5518 r_squared_test: 0.5633
Iteration: 200/2000 [-----] Loss: 4.50e+07 Validation_Loss: 1.146930e+07 r_squared: 0.5843 r_squared_test: 0.5972
Iteration: 300/2000 [-----] Loss: 4.46e+07 Validation_Loss: 1.135233e+07 r_squared: 0.5882 r_squared_test: 0.6013
Iteration: 400/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133659e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 500/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133445e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 600/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133415e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 700/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 800/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 900/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 1000/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 1100/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019
Iteration: 1200/2000 [-----] Loss: 4.45e+07 Validation_Loss: 1.133411e+07 r_squared: 0.5888 r_squared_test: 0.6019

Early stopping at Iteration: 1291 Validation_Loss: 11334106.5172 Validation_r_squared: 0.60194

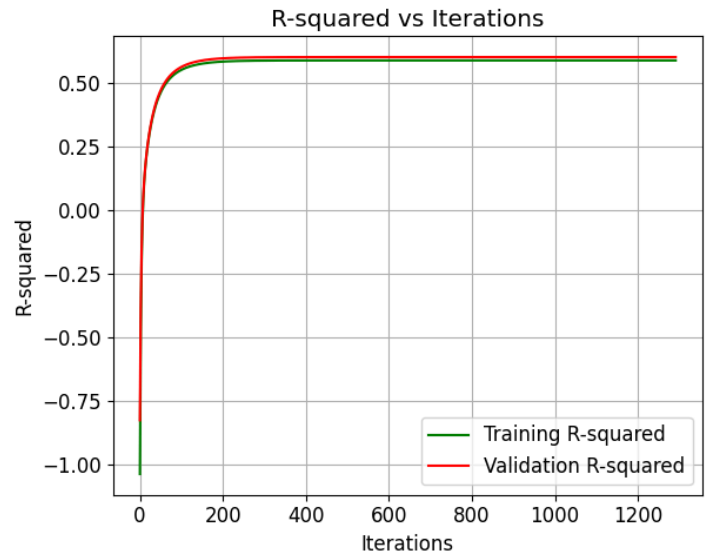
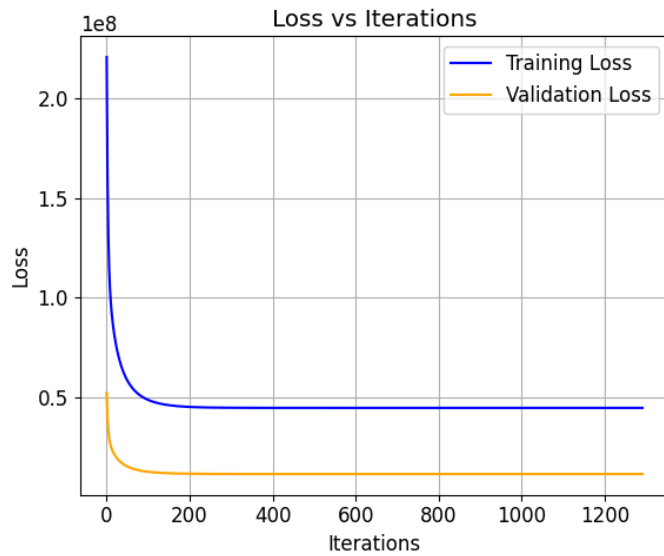
The Final Values of m and b are: [ 5163.57645849  3137.57686678  1683.26350188  2580.59864961
 3097.01704074 -3747.83290909  9425.44859943  1738.35908969
  790.07542264  2077.10411472  1074.07706331] 5676.615690343749

```

https://colab.research.google.com/drive/1TfoU0VJxU51qHbXqiyqVQoWHAMygaKdM#scrollTo=bu_kn2Svt9mm&printMode=true 15/17

```
plt.ylabel('R-squared')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(22,5))
```

```
plt.subplot(1,3,1)
plt.plot(iterations, LR)
plt.title('Learning Rate vs Iterations')
plt.xlabel('Iterations')
plt.ylabel('Learning Rate')
plt.grid(True)
```

```
plt.subplot(1,3,2)
plt.plot(iterations, Intercept)
plt.title('Intercept vs Iterations')
plt.xlabel('Iterations')
plt.ylabel('Intercept')
plt.grid(True)
```

```
plt.subplot(1,3,3)
plt.plot(iterations, Slope)
plt.title('Slope vs Iterations')
plt.xlabel('Iterations')
plt.ylabel('Slope')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```